HANDS_ON_dependencies_EN

Hands-on: Job Dependencies in Apache NiFi (English)

> Apache NiFi 2.8.0, single-user login (`admin`), UI at https://localhost:8443/nifi German version: `HANDS_ON_Abhaengigkeiten.md`. Simpler file-to-file track: `HANDS_ON_file_to_file.md`.

How does NiFi express a “dependency”?

NiFi is **not** a classic job scheduler (no “run Job A, then Job B” button). A processor only becomes active when a **FlowFile sits in its incoming queue**. So “job control” in NiFi means: control when a FlowFile lands in the next processor’s queue. There are four ways:

Mechanism	“B starts when …”	Lab
Data dependency (connection / relationship)	… A produced a FlowFile on the relationship	Lab 1
Signal (Wait / Notify)	… A has set a named signal	Lab 2
Barrier / fan-in (MergeContent, Wait counter)	… N inputs are present	Lab 3
Time + condition (cron scheduling, RouteOnAttribute)	… a time is reached / a condition holds	Lab 4

Common paths (copy/paste)

```
Input:    /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/input
Output:   /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/
output
Reports:  /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/
reports
Archive:  /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/
archive
Samples:  /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/
samples
DB:       /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/db/
fmea.db
Driver:   /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/drivers/
sqlite-jdbc-3.45.1.0.jar
```

One-time DB setup for the orders labs

```
sqlite3 <DB> "CREATE TABLE IF NOT EXISTS orders (order_id INTEGER, datum TEXT, menge
INTEGER, betrag REAL);"
```

Generated sample data (in `data/samples/`)

File	Purpose
`orders.csv` (5 rows)	main dataset for Lab 1 / Lab 2
`customers.csv`, `products.csv`	the three "jobs" for the fan-in Lab 3
`sensor_ok.csv`	small file for the file-to-file track
`orders_broken.csv`	**deliberately broken** (bad types, wrong column counts) to trigger the failure path

---

Lab 1: Data dependency: "load first, then report success"

Goal: the `success` relationship is the simplest dependency: the downstream processor runs only if the previous one succeeded. The `failure` relationship is the other dependency edge.

Flow:

```
GetFile ──success──> PutDatabaseRecord ──success──> PutFile (success marker)
                              └──────failure──────> PutFile (error folder)
```

Config: 1. `GetFile`: Input Directory = Input path, File Filter = `.*\.csv`. 2. `PutDatabaseRecord`: Record Reader = CSVReader, DB Pool = DBCPConnectionPool, Statement Type = `INSERT`, Table Name = `orders`. 3. `PutFile` "success": Directory = Reports path. 4. `PutFile` "error": Directory = `Output/errors`. 5. Connect `success` -> success-PutFile, `failure` -> error-PutFile. Terminate leftover relationships.

Verify (happy path):

```
cp <Samples>/orders.csv <Input>/
ls -l <Reports>/                                  # marker present
sqlite3 <DB> "SELECT COUNT(*) FROM orders;"       # 5
```

Trigger the failure path (the point of Lab 1):

```
cp <Samples>/orders_broken.csv <Input>/
ls -l <Output>/errors/                            # broken file routed here, not
silently dropped
```

Discussion: Why is "auto-terminate failure" a bad idea in production? What does the failure edge buy you over just ignoring errors?

Lab 2: Wait / Notify: real "B after A" across independent flows

Goal: couple two independent flows so Job B starts only after Job A finishes, without B receiving A's FlowFiles directly. This is NiFi's equivalent of "B depends on A".

Controller services (NiFi 2.x names!) : create once, enable both: > In the Add dialog filter for `MapCache`. The 2.x names are `MapCacheServer` and > `MapCacheClientService` (the old `DistributedMapCache*` names from older tutorials are gone). 1. `MapCacheServer` (defaults, port

4557) -> enable. 2. `MapCacheClientService`: Server Hostname = `localhost`, Server Port = `4557` -> enable.

Flow:

```
Job A:  GetFile ──> PutDatabaseRecord ──success──> Notify   (signal: orders-loaded)
Job B:  GenerateFlowFile ──> Wait ──success──> ExecuteSQL ──> PutFile (report)
                              └ Wait blocks until signal "orders-loaded" exists
```

Notify (Job A): Release Signal Identifier = `orders-loaded`; Distributed Cache Service = the client; terminate `success`.

Wait (Job B): Release Signal Identifier = `orders-loaded`; Distributed Cache Service = same client; Target Signal Count = `1`; `success` -> ExecuteSQL; `expired` -> terminate (or PutFile "timeout"); `failure` -> terminate.

ExecuteSQL (Job B): DB Pool = DBCPConnectionPool; SQL select query = `SELECT COUNT(*) AS anzahl FROM orders`. Then `PutFile` -> Reports.

GenerateFlowFile (Job B trigger): Scheduling tab -> Run Schedule = `10 sec`.

The "aha" sequence: 1. Start **only Job B**. FlowFiles pile up in the Wait self-loop queue: Wait blocks. Nothing happens. 2. Now start **Job A** and drop `orders.csv`. Notify fires -> Wait releases -> the report is written.

> **Practical notes (verified in a live test):** 1. `wait` **relationship must self-loop:** Drag from Wait back onto Wait, relationship `wait`. Otherwise waiting FlowFiles disappear. Auto-terminate ONLY `expired` and `failure`, NOT `wait`. 2. **~30s patience:** FlowFiles on the `wait` loop are penalized (default 30s), so release after the Notify signal can take up to 30s. Do not declare failure too early. 3. **Reset the signal between runs:** the signal stays in the MapCache. For a clean re-run, briefly disable and re-enable the `MapCacheServer`. 4. **ExecuteSQL pitfall:** in NiFi 2.8 ExecuteSQL triggered without a real incoming FlowFile can throw `IllegalStateException: Already added pointer to statements set`. For the pure Wait/Notify demo, use a **PutFile** as Job B's terminal node (writes a marker file on release) instead.

```
ls -l <Reports>/
```

Discussion: What happens to the signal after the first release? How would you make "B runs exactly once per A run"? What is the `expired` relationship (timeout) for?

Lab 3: Barrier / fan-in: "start only when all three files arrived"

Goal: build a synchronization barrier: a step runs only when several preconditions are met.

Scenario: `orders.csv`, `customers.csv`, `products.csv` should be processed together only once all three have arrived.

Variant A (simple, MergeContent):

```
GetFile ──> MergeContent ──merged──> PutFile (bundle)
```

- `MergeContent`: Minimum Number of Entries = `3`, Maximum Number of Entries = `3`, Max Bin Age = `30 sec` (safety valve).
- While only 1 or 2 files are present the bin stays open: no output. The third file triggers `merged`.

Variant B (advanced, Wait counter): - Each incoming file calls `Notify` with `Signal Counter Name = eingang`. - `Wait` with `Target Signal Count = 3` and the same counter releases only after the third signal. - Advantage over MergeContent: the contents stay separate FlowFiles; only the release is synchronized.

Verify:

```
cp <Samples>/orders.csv     <Input>/
cp <Samples>/customers.csv  <Input>/
cp <Samples>/products.csv   <Input>/
ls -l <Output>/             # bundle appears only after the third file
```

Discussion: What does `Max Bin Age` do and why is it an important safety valve? How does “wait for any 3” differ from “wait for 3 specific ones”? (hint: `Correlation Attribute Name`)

Lab 4: Time scheduling + conditional routing

Goal: learn time-based triggering (cron) and data-dependent branching as two more dependency forms.

Part A: cron / timer trigger - `GetFile` -> Scheduling tab: Scheduling Strategy = `CRON driven`, e.g. `*/30 * * * * ?` (every 30 s). Alternative: `Timer driven` with Run Schedule = `30 sec`. - Discuss: Timer driven (interval) vs CRON driven (wall-clock / calendar).

Part B: conditional routing with RouteOnAttribute

```
GetFile ──> RouteOnAttribute ──is_orders──> PutDatabaseRecord (table orders)
                             └─unmatched──> PutFile (Output/other)
```

- `RouteOnAttribute`: add property `is_orders` = `${filename:startsWith("orders")}`, Routing Strategy = `Route to Property name`.
- `is_orders` -> PutDatabaseRecord; `unmatched` -> PutFile.

Verify:

```
cp <Samples>/orders.csv    <Input>/   # -> DB
cp <Samples>/sensor_ok.csv <Input>/   # -> Output/other
ls -l <Output>/other/
sqlite3 <DB> "SELECT COUNT(*) FROM orders;"
```

Discussion: When is cron better than pure data-triggering (GetFile already polls)? How would you combine multiple conditions (filename and size)?

Reset between labs

```
sqlite3 <DB> "DELETE FROM orders;"
rm -f <Input>/* <Output>/* <Reports>/*
# reset Wait/Notify signals: disable + re-enable MapCacheServer
```

Important: GetFile polling interval (~30s patience)

`GetFile` defaults to **Polling Interval = 30 sec**. After you start it (or drop a file in), pickup can take **up to 30 seconds**. It briefly looks like “nothing happens” : everything is fine, it just polls on the next cycle. In class, wait **30-40s** after dropping a file before debugging. To speed it up, set GetFile’s **Polling Interval** to e.g. `2 sec`.

Common pitfalls

- **PutDatabaseRecord invalid:** DBCPConnectionPool empty or not enabled (check URL + driver class + driver location).
- **Wait never releases:** Notify uses a different Release Signal Identifier, or a different cache client than Wait. Both must match exactly.
- **Duplicate rows:** GetFile with `Keep Source File = true` re-ingests the same file. Use `false` for clean tests.
- **MergeContent emits nothing:** fewer entries than `Minimum Number of Entries`, and `Max Bin Age` not yet reached.

Teaching arc

1. Lab 1: a dependency is just a connection (`success`/`failure`).
2. Lab 2: decoupling two jobs via signals (Wait/Notify) = “B after A”.
3. Lab 3: synchronization barrier (fan-in) = “B after several A”.
4. Lab 4: time- and condition-driven triggering.